



Grundlagen

Vom relationalen Modell zur Vektordatenbank

Relationale Datenbanken: Aufbau

Daten sind strukturiert, tabellarisch und über Relationen verknüpft.

Tabelle: Kunden

KundenID	Name	Stadt
1	Anna Müller	Berlin
2	Ben Schmidt	München
3	Clara Weber	Hamburg

Tabellen

Daten in Zeilen und Spalten – jede Zeile ein Datensatz, jede Spalte ein Attribut

Tabelle: Bestellungen

BestellID	KundenID	Produkt
101	1	Laptop
102	1	Maus
103	2	Tastatur

Geordnet

Jedes Feld hat einen festen Typ: Text, Zahl, Datum – keine freien Strukturen

Relationen

Tabellen sind über Schlüssel verbunden – ermöglicht komplexe, verschachtelte Abfragen

Relationale Datenbanken: Stärken & Schwächen

Stärken

✓ **Exakte Suche**
WHERE preis = 49.99 — findet genau diesen Wert, immer.

✓ **Filtern über Relationen**
JOIN über Tabellen — alle Bestellungen eines Kunden aus Berlin.

✓ **Konsistenz & Integrität**
Transaktionen, ACID-Garantien, keine inkonsistenten Zustände.

Schwächen

✗ **Unstrukturierte Daten**
Bilder, Audiodateien, Fließtext — kein festes Schema, kein natürlicher Platz.

✗ **Ähnlichkeitssuche**
SQL kennt kein "ähnlich wie" — nur exakte Werte oder Ranges.

Genau diese Schwächen löst eine Vektordatenbank.

Was sind Vektordatenbanken?

Eine Datenbank, die für unstrukturierte Daten und Ähnlichkeitssuche gebaut ist.



Unstrukturierte Daten

Texte, Bilder, Audio, Video — keine festen Schemas erforderlich. Die Daten müssen nicht in Tabellen passen.



Ähnlichkeitssuche

Statt "exakt gleich" findet sie "semantisch ähnlich" — das nächste Bild, den ähnlichsten Text, das relevanteste Dokument.



Kein manuelles Labeln

Embedding-Modelle extrahieren Bedeutung automatisch. Menschen müssen Daten nicht kategorisieren.



Vektoren als Repräsentation

Jedes Objekt wird als Zahlenvektor im hochdimensionalen Raum dargestellt. Ähnlichkeit = geometrische Nähe.

Embedding-Modelle: Überblick

Wie werden Texte, Bilder oder Audio in Vektoren umgewandelt?

Frequenzbasiert

Basiert auf dem Zählen von Wort-Vorkommen und -Häufigkeiten

Co-Occurrence Matrix

TF-IDF

Vorhersagebasiert

Lernt Wortbedeutungen aus dem Kontext durch neuronale Netze

Word2Vec (CBOW / SkipGram)

GloVe

Kontextuell

Co-Occurrence Matrix

Kernidee: Wörter, die oft zusammen auftreten, haben ähnliche Bedeutung.

- Zählt Wort-Paar-Auftritte in einem festen Kontext-Fenster
- Erzeugt eine Matrix der Größe Vokabular $\times$ Vokabular
- Jede Zeile repräsentiert ein Wort als Vektor
- Einfache Repräsentation — aber Dimension wächst mit dem Vokabular
- Bsp: “Hund isst Fleisch”, “Katze isst Fisch”, “Katze isst Fleisch”

Beispiel: Fenster = 2

	Hund	isst	Fleisch	Katze	Fisch
Hund	0	1	1	0	0
isst	1	0	2	2	1
Fleisch	1	2	0	1	1
Katze	0	2	1	0	1
Fisch	0	1	0	1	0

TF-IDF – Term Frequency · Inverse Document Frequency

Kernidee: Wichtigkeit eines Wortes im Dokument relativ zum gesamten Korpus bestimmen.

TF – Term Frequency (lokal)

Wie oft kommt ein Wort im aktuellen Dokument vor?

$TF(t, d) = \text{Anzahl von } t \text{ in } d / \text{Gesamtwörter in } d$

→ Häufiges Wort im Dokument = hoher TF-Wert

IDF – Inverse Document Frequency (global)

Wie selten ist das Wort im gesamten Korpus?

$IDF(t) = \log(N / df(t))$

→ Seltenes Wort im Korpus = hoher IDF-Wert

TF-IDF = $TF \times IDF$ — stark bei seltenen, aussagekräftigen Begriffen. Schwach bei Stoppwörtern wie „der“, „ist“, „und“.

Beispiel:

In einem Medizin-Dokument: „Aspirin“ hat hohen TF (oft genannt) und hohen IDF (selten in anderen Dokumenten) → hoher TF-IDF Score. „Der“ hat hohen TF aber niedrigen IDF → niedriger Score.

Word2Vec – CBOW & SkipGram

Kernidee: Vorhersage-basierte Embeddings durch neuronale Netze.

CBOW

Continuous Bag of Words

Kontext → Zielwort

„Der ____ bellt laut"
→ Vorhersage: „Hund“

Nutzt mehrere Kontextwörter gleichzeitig.

SkipGram

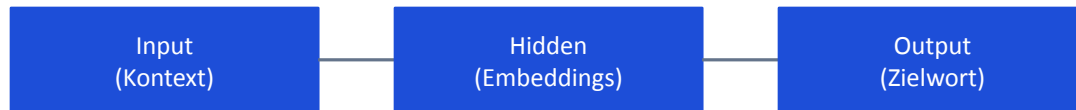
Zielwort → Kontext

„Hund"
→ Vorhersage: „bellt“, „treues“, „Tier“

Besonders gut bei seltenen Wörtern.

Die Embeddings sind die trainierten Gewichte des neuronalen Netzes — nicht das Ergebnis der Vorhersage.

Neuronales Netz (vereinfacht):



GloVe – Global Vectors for Word Representation

Kernidee: Das Beste aus beiden Welten — globale Co-Occurrence-Statistik + effiziente Vektoren.

- Nutzt globale Co-Occurrence-Statistik des gesamten Korpus — nicht nur lokale Kontext-Fenster
- Matrix-Faktorisierung auf Log-Wahrscheinlichkeiten der Kookkurrenzen
- Skaliert exzellent auf große Datensätze
- Kombination: statistische Stärke von Co-Occurrence Matrix + Effizienz von Word2Vec

Vergleich

	Lokal	Global
Co-Occ.	✓	✗
Word2Vec	✓	✗
GloVe	✓	✓

Beispiel:

GloVe lernt: $P(\text{Eis} \mid \text{Wasser})$ ist viel höher als $P(\text{Feuer} \mid \text{Wasser})$. Diese Verhältnisse aus dem gesamten Korpus kodiert es direkt in die Vektoren — präziser als lokale Fenster allein.

Semantische Ähnlichkeit

Ähnlichkeit ist Distanz im Vektorraum.

- Jedes Wort, Dokument oder Objekt wird als Punkt im hochdimensionalen Raum dargestellt
- Semantisch ähnliche Konzepte liegen geometrisch nah beieinander
- Die Distanz zwischen zwei Punkten = Maß für Unähnlichkeit
- Die Richtung des Vektors trägt die semantische Information



Kosinusähnlichkeit

Misst den Winkel zwischen zwei Vektoren — nicht ihre Länge.

Berechnung & Wertebereich

Wertebereich: -1 bis $+1$

$+1$ = gleiche Richtung (sehr ähnlich)

0 = orthogonal (kein Zusammenhang)

-1 = entgegengesetzt

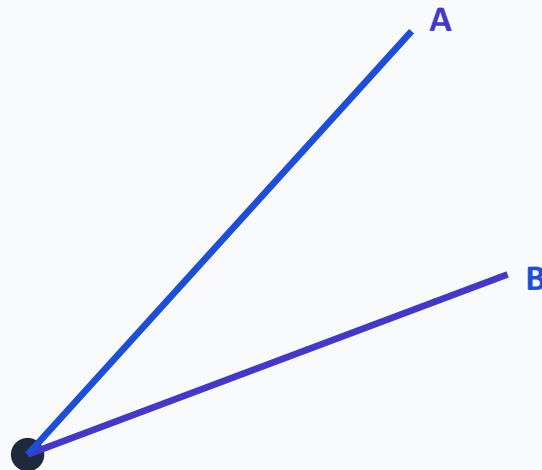
Besonderheit

Ignoriert die Länge der Vektoren komplett — nur die Richtung zählt. Ein kurzes und ein langes Dokument über dasselbe Thema haben hohe Kosinusähnlichkeit.

Typischer Einsatz

Dokumentenähnlichkeit, Texte unterschiedlicher Länge vergleichen, Empfehlungssysteme.

Winkel θ zwischen A und B



Euklidische Distanz (L2)

Misst die direkte "Luftlinie" zwischen zwei Punkten im Vektorraum.

Berechnung

Kleine Distanz = ähnlich Große Distanz = unähnlich

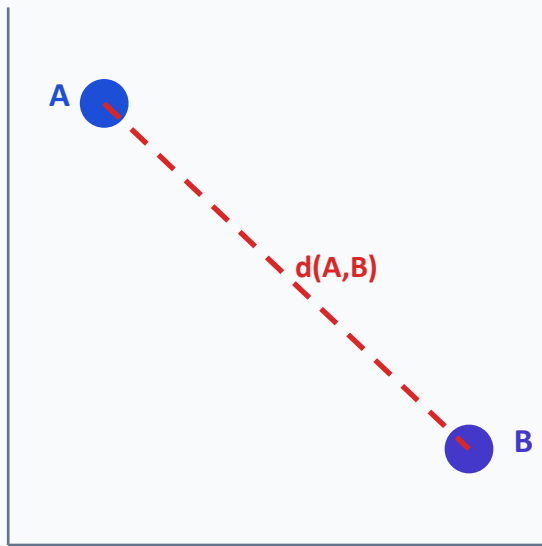
Unterschied zu Kosinus

L2 berücksichtigt sowohl Richtung als auch Länge. Zwei Vektoren mit gleicher Richtung aber unterschiedlicher Länge haben eine L2-Distanz größer 0.

Typischer Einsatz

Bilder, Farben, Feature-Vektoren aus Computer Vision. Überall dort, wo die Magnitude des Vektors eine inhaltliche Bedeutung hat.

L2-Distanz im 2D-Raum



Skalarprodukt (Dot Product)

Effizient, skalierbar — und bei normalisierten Vektoren äquivalent zu Kosinus.

Berechnung

Einfache elementweise Multiplikation und Summe

Berücksichtigt

Sowohl Länge als auch Winkel — ähnlich wie L2, aber andere Formel.
Große Werte = ähnlich (wenn Vektoren normalisiert sind).

Besonderer Vorteil

Rechnerisch günstiger als Kosinus, da keine Division durch Längen nötig.
→ Ideal für Large-Scale und Echtzeit-Systeme.
→ Bei normalisierten Vektoren: Dot Product = Kosinus-Ähnlichkeit.

Vergleich der drei Metriken

Metrik	Länge	Winkel
Kosinus	✗	✓
L2	✓	✓
Dot	✓	✓

Faustregel: Kosinus für Text, L2 für Bilder, Dot für Geschwindigkeit.



Die Architektur

Wie ist eine Vektordatenbank aufgebaut?

Das Grundprinzip: Trennung von Verantwortlichkeiten

Warum eine Schichtenarchitektur?

- Moderne Vektordatenbanken bestehen aus vier klar getrennten Schichten
- Jede Schicht trägt eine eigene, abgegrenzte Verantwortung
- Die Trennung ist eine bewusste Designentscheidung
- Ziel: Unabhängige Skalierbarkeit, Wartbarkeit und Fehlertoleranz

Kernprinzip

Wenn jede Schicht unabhängig arbeitet, kann sie auch unabhängig skalieren, ausfallen und optimiert werden.

Überblick: Die vier Schichten

Dienstschicht

Verbindungen · Routing · Sicherheit · Multi-Tenancy

Abfrageschicht

Anfragen verarbeiten · Strategie wählen · Caching

Indexschicht

Indexierungsalgorithmen · Hardware-Optimierung

Speicherschicht

Persistente Datenhaltung · Komprimierung · I/O

Schicht 1: Speicherschicht

▼ Unterste Schicht – Fundament

- Persistente Speicherung aller Vektoren und zugehöriger Metadaten
- Spezialisierte Kodierung und Komprimierung für hochdimensionale Daten
- Optimierte I/O-Muster für vektorspezifische Zugriffsmuster
- RAM für schnellen Zugriff — NVMe-SSD für kosteneffiziente Skalierung (DiskANN)

Maßstab

100 Mio. Vektoren
× 1.024 Dimensionen
× 4 Byte (float32)
= ~400 GB RAM

→ Ohne SSD-Indexierung nicht wirtschaftlich skalierbar.

Schlüsselwort: Spezialisierung – kein Allzweck-Speicher

Schicht 2: Indexschicht

▼ Strukturierung für schnellen Zugriff

Was ist ein Index?

Eine vorberechnete Datenstruktur über die gespeicherten Vektoren. Sie speichert keine Vektoren selbst — das macht die Speicherschicht. Sie speichert eine Karte: Welche Vektoren liegen nah beieinander? Wenn eine Anfrage kommt, folgt die Suche dieser Karte statt alles zu durchsuchen.

- Verwaltet mehrere Indexierungsalgorithmen gleichzeitig – je nach Anwendungsfall
- Wird fortlaufend aktualisiert, wenn neue Daten eintreffen
- Hardwarespezifische Optimierungen: (AVX512, SIMD-Instruktionen, GPU-Beschleunigung)

Algorithmen (Vorschau)

HNSW · IVF · PQ · DiskANN

→ Im Suchalgorithmen Teil erklärt

Warum Vektorsuche grundlegend schwierig ist

Das Problem: Hohe Dimensionalität heißt viele Daten

Exakte Suche (kNN)

Bei 1 Milliarde Vektoren:
Jede Anfrage vergleicht den Query-Vektor mit ALLEN gespeicherten Vektoren.

- 1 Mrd. Berechnungen pro Anfrage
- Bei 1.024 Dimensionen: Milliarden von Multiplikationen
- Nicht in Echtzeit möglich

Approximate Nearest Neighbor (ANN)

Statt exakter Suche: intelligente Abkürzungen.

Der Index strukturiert Vektoren vorab so, dass man nur einen kleinen Bruchteil durchsuchen muss.

- Millisekunden statt Minuten
- Kleine Genauigkeitseinbußen (akzeptabel)
- Das ist die Aufgabe der Indexschicht

Schicht 3: Abfrageschicht

▼ Das Gehirn jeder Suchanfrage

- Nimmt eingehende Suchanfragen entgegen und parst sie
- Bestimmt die Ausführungsstrategie – welcher Index, welche Partition
- Verarbeitet, bewertet und rankt die zurückgegebenen Ergebnisse
- Implementiert Caching für häufig wiederkehrende Anfragen

Warum Caching?

1.000 Nutzer suchen dasselbe Produkt → das Ergebnis wird nicht 1.000× neu berechnet.

Schicht 4: Dienstschicht

▲ Oberste Schicht – Schnittstelle zur Außenwelt

Verbindungen

API-Endpunkte, Client-Verwaltung,
Load Balancing

Monitoring

Logging, Metriken, Alerts bei
Systemfehlern

Sicherheit

Authentifizierung, Autorisierung,
Verschlüsselung

Multi-Tenancy

Verschiedene Teams oder Kunden teilen dieselbe Datenbankinstanz – ohne gegenseitigen Datenzugriff.
Realisiert durch Sharding, Partitionierung oder Row-Level Security.

Welche dieser vier Schichten haltet ihr für die kritischste — und warum?

Denkanstoß: Denkt daran: Was passiert, wenn genau diese eine Schicht ausfällt oder falsch gebaut ist?

Warum diese Trennung? – Das Prinzip der Disaggregation

Compute und Storage sind entkoppelt

- Monolithische Systeme: Alles skaliert zusammen – teuer und unflexibel
- Schichtenarchitektur: Jede Einheit skaliert nur dort, wo Bedarf besteht
- Kein Ausfall einer Schicht bringt das gesamte System zum Stillstand

Mehr Speicher?

→ Nur Speicherschicht erweitern

Mehr Anfragen?

→ Nur Abfrageschicht skalieren

Neue Indizes?

→ Nur Indexschicht aktualisieren

Das ermöglicht es, Milliarden von Vektoren in Produktionssystemen zu verwalten – ohne Kompromisse bei der Performance.

Disaggregation in der Praxis: Ein konkretes Szenario

Stellt euch vor: Ein Startup, das gerade viral geht.

01

Das Problem

Euer monolithischer Server hat 64 GB RAM. Ihr habt 50 Mio. Vektoren gespeichert – der Speicher ist fast voll. Gleichzeitig explodiert die Anzahl der Anfragen.

02

Monolithische Lösung

Ihr kauft einen größeren Server: 256 GB RAM, doppelt so viele CPU-Kerne – auch wenn das CPU-Problem gar nicht existiert. Kosten verdoppeln sich.

03

Disaggregierte Lösung

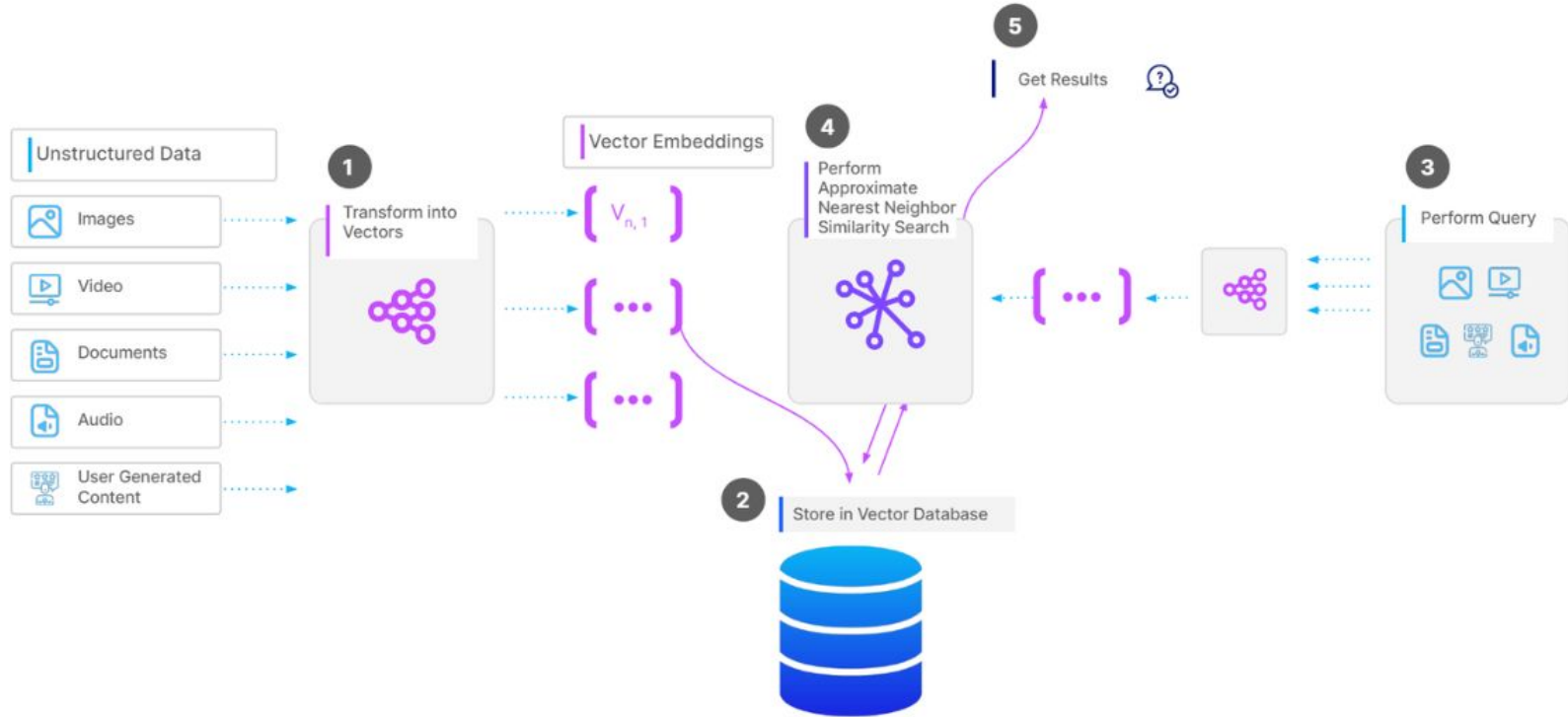
Ihr fügt einen Speicherknoten hinzu. Abfrage- und Indexknoten bleiben unverändert. Nur das Problem wird gelöst – und nur dort, wo es existiert.



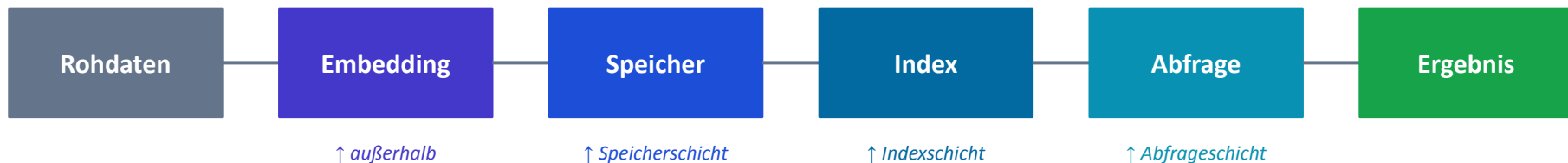
Der Workflow einer Vektorsuche

Von Rohdaten zum Suchergebnis — Schritt für Schritt

Kompletter Workflow einer Vector Suchoperation



Die Pipeline: Architektur in Bewegung



1

Embedding → **außerhalb der DB**

Rohdaten werden in Vektoren umgewandelt – durch ein Modell wie BERT oder CLIP – bevor sie die Datenbank erreichen. Die DB nimmt fertige Vektoren entgegen.

2

Speicher + Index → **Speicher- & Indexschicht**

Der Vektor wird persistent gespeichert (Speicherschicht) und gleichzeitig in den ANN-Index eingetragen (Indexschicht). Beide Schritte sind eng koordiniert.

3

Abfrage → **Abfrageschicht + Dienstschicht**

Eine Suchanfrage trifft über die Dienstschicht ein, wird von der Abfrageschicht verarbeitet, und das Ergebnis wird über dieselbe Dienstschicht zurückgeliefert.

Schritt 1: Transformation & Embedding

Außerhalb der Datenbank — vor dem Eintritt in das System



Was passiert hier?

- Rohdaten: Text, Bild, Audio, Video; werden einem Embedding-Modell übergeben
- Das Modell (z.B. BERT für Text, CLIP für Bilder) wandelt das Objekt in einen Zahlenvektor um
- Dieser Vektor repräsentiert die semantische Bedeutung des Objekts
- Erst dieser fertige Vektor wird an die Vektordatenbank übergeben, nicht die Rohdaten



Modellwechsel = Neueinbettung aller Daten

Wechselt das Embedding-Modell, sind alle gespeicherten Vektoren ungültig. Der gesamte Datensatz muss neu eingebettet werden.

Beispiel: Text-Embedding

"Der Hund bellt laut"

↓ BERT

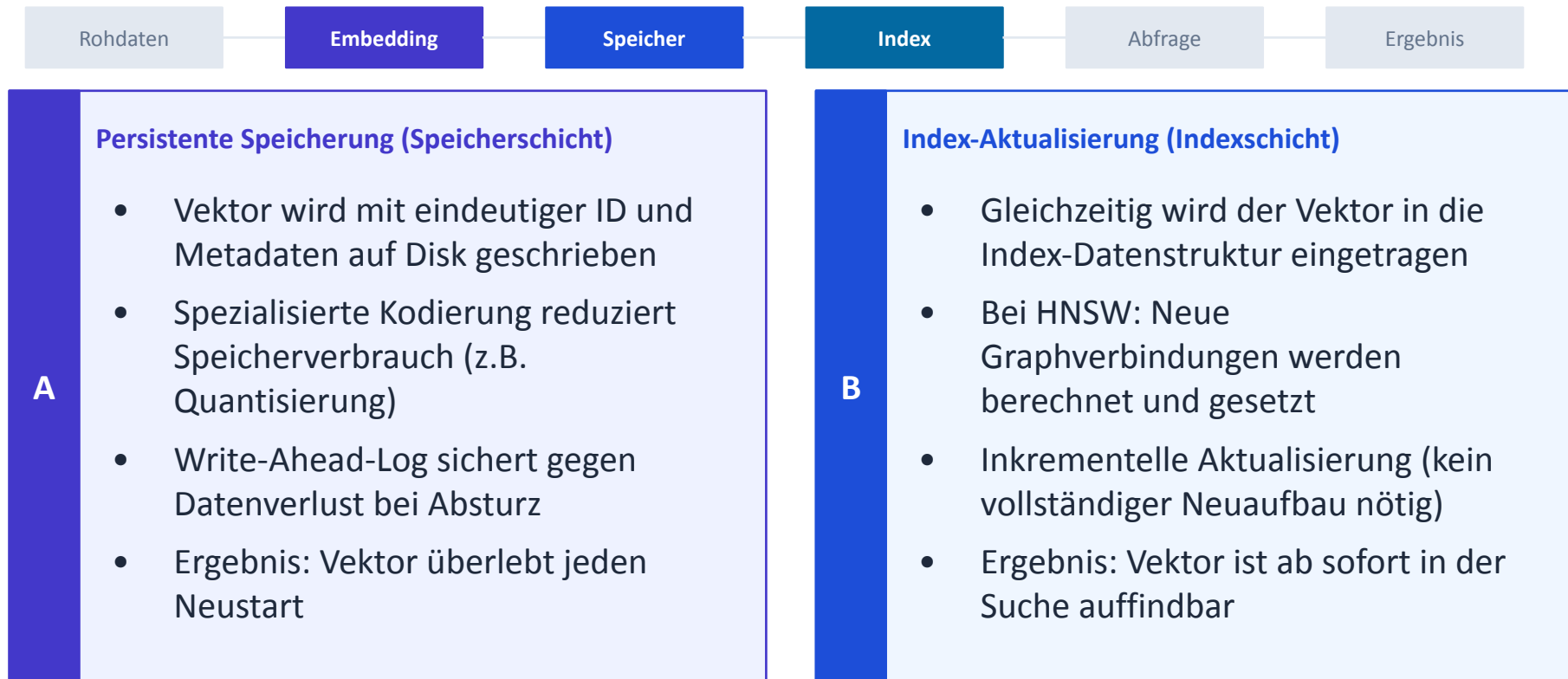
[0.21, -0.84, 0.37,
0.92, -0.15, 0.63,
... (768 Werte)]

↓ an die DB

Vektordatenbank

Schritt 2: Speicherung & Indexierung

Speicherschicht + Indexschicht arbeiten koordiniert



Beide Schritte passieren beim Einfügen eines neuen Vektors — koordiniert und transaktional.

Schritt 3: Abfrageverarbeitung

Abfrageschicht + Dienstschicht — wenn eine Suchanfrage eintrifft

Rohdaten

Embedding

Speicher

Index

Abfrage

Ergebnis

1

Anfrage trifft über die Dienstschicht ein

Client sendet einen Query-Vektor — embedded mit demselben Modell wie die gespeicherten Daten. Anderes Modell = anderer Vektorraum = falsche Ergebnisse.

2

Abfrageschicht wählt eine Strategie

Welcher Index wird verwendet? Welche Partition ist relevant? Gibt es Metadaten-Filter? Ist das Ergebnis bereits gecacht?

3

ANN-Suche wird ausgeführt

Der Index (HNSW, LSH, etc.) traversiert seine Datenstruktur und liefert Kandidaten zurück — die K ähnlichsten Vektoren aus dem relevanten Suchraum.

4

Ergebnisse werden assembliert

Kandidaten werden mit Ähnlichkeitswerten versehen, ggf. nach Metadaten gefiltert (Post-Filtering), gerankt und als Antwort vorbereitet.

Schritt 4: Post Processing & ErgebnISRückgabe

Von der Kandidatenliste zum ausgelieferten Ergebnis

Rohdaten

Embedding

Speicher

Index

Abfrage

Ergebnis

Ähnlichkeitsbewertung

Jeder Kandidat erhält einen Score □ Distanzwert aus der ANN-Suche.

Niedrigere Distanz = höherer Score = relevanteres Ergebnis.

Post-Filtering

Metadaten-Filter werden auf die Kandidatenliste angewendet: Preistrage, Kategorie, Datum. Treffer ohne passende Metadaten fallen heraus.

Ranking & Sortierung

Die verbleibenden Kandidaten werden nach Score geordnet. Optional: Reranking durch ein zweites Modell für höhere Relevanz.

Rückgabe über Dienstschrift

Das finale Ergebnis: Vektoren, IDs, Metadaten, Scores
Wird serialisiert und über die API an den Client zurückgeliefert.



Suchalgorithmen

Wie findet eine Vektordatenbank ähnliche Einträge?

Überblick: Suchalgorithmen

Von exakt zu approximativ — und von einfach zu produktionstauglich.



Erweiterungen & Kombinationen:

Filtered Search

Vektorsuche kombiniert mit
Metadaten-Filtern

Full-Text Search

Keyword-Suche + semantische
Vektorsuche kombiniert

K-Nearest Neighbor (KNN)

Der naive Ansatz — und der genaueste.

Wie es funktioniert

Gegeben ein Query-Vektor: Berechne die Distanz zu ALLEN gespeicherten Vektoren. Gib die K nächsten zurück.

Vorteile



Perfekte Genauigkeit

Gibt garantiert die K tatsächlich nächsten Vektoren zurück.

Nachteile



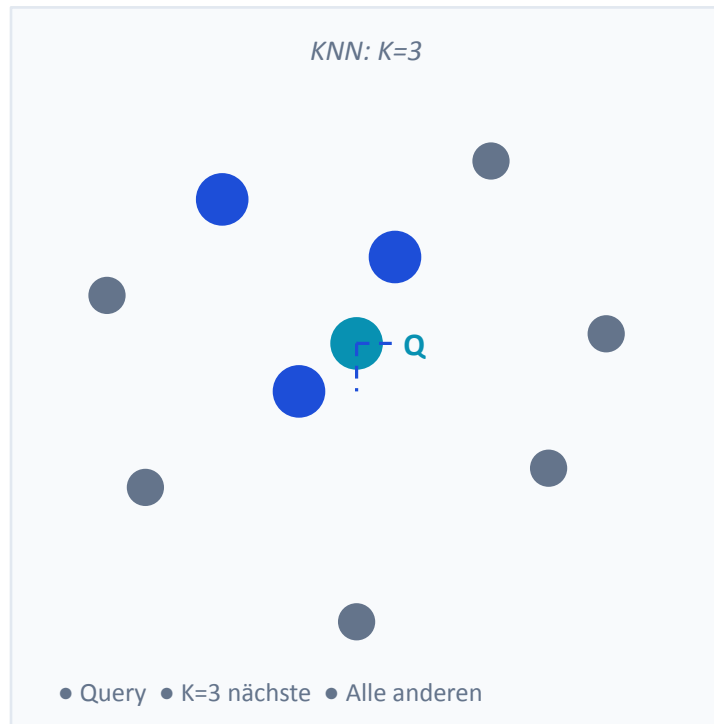
Lineare Komplexität

$O(n)$ — jede Anfrage vergleicht alle n Vektoren. Bei Millionen Einträgen: Sekunden bis Minuten.



Nicht skalierbar

Je mehr Daten, desto langsamer — ohne Ausnahme.



Approximate Nearest Neighbor (ANN)

Schneller als KNN — mit kontrolliertem Genauigkeitsverlust.

Kernidee

Statt alle Vektoren zu vergleichen: Durchsuche nur eine intelligente Teilmenge. Die Ergebnisse sind nicht garantiert exakt — aber in der Praxis gut genug.

Genauigkeit ← ————— Kompromiss ————— → Geschwindigkeit

KNN — Exakt

Genauigkeit: 100%

Geschwindigkeit: Langsam

Praktisch nicht nutzbar bei Millionen Einträgen

ANN — Approximativ

Genauigkeit: ~95–99%

Geschwindigkeit: Sehr schnell

Millisekunden — selbst bei Milliarden Einträgen

HNSW und LSH sind konkrete Implementierungen von ANN.

HNSW – Hierarchical Navigable Small World

Ein mehrschichtiger Graph, der von grob nach fein sucht.

Aufbau

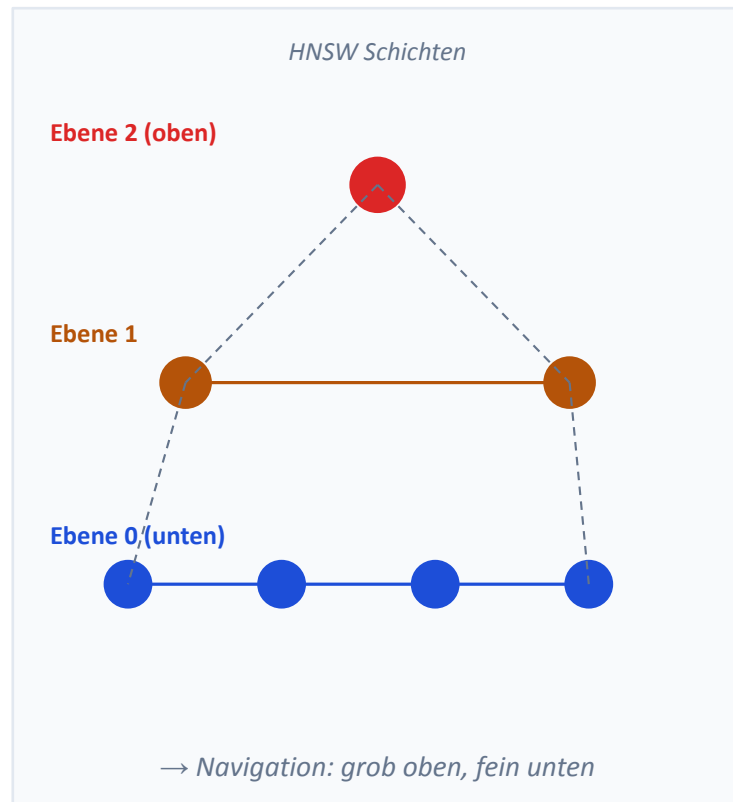
Vektoren werden als Knoten in einem mehrschichtigen Graphen gespeichert. Jeder Knoten ist mit ähnlichen Knoten verbunden — die Verbindungsdichte nimmt nach unten zu.

Suche

Start in der obersten Schicht (wenige Knoten, grobe Navigation).
Springe Ebene für Ebene nach unten, verfeinere dabei die Suche.
Sehr schnell bei hoher Genauigkeit.

Einsatz

Beliebtester ANN-Algorithmus in Produktionssystemen. Standard in Milvus, Pinecone, Weaviate.



LSH – Locality-Sensitive Hashing

Ähnliche Vektoren landen mit hoher Wahrscheinlichkeit im selben Hash-Bucket.

Grundprinzip

Wende eine Hash-Funktion an, die ähnliche Vektoren mit hoher Wahrscheinlichkeit auf denselben Bucket abbildet. Suche dann nur in diesen Buckets — nicht im gesamten Datensatz.

Wie es sich von HNSW unterscheidet

HNSW nutzt einen Graphen für Navigation. LSH nutzt Hash-Funktionen für Gruppierung. Beide sind ANN — mit unterschiedlichen Stärken.

Vorteile

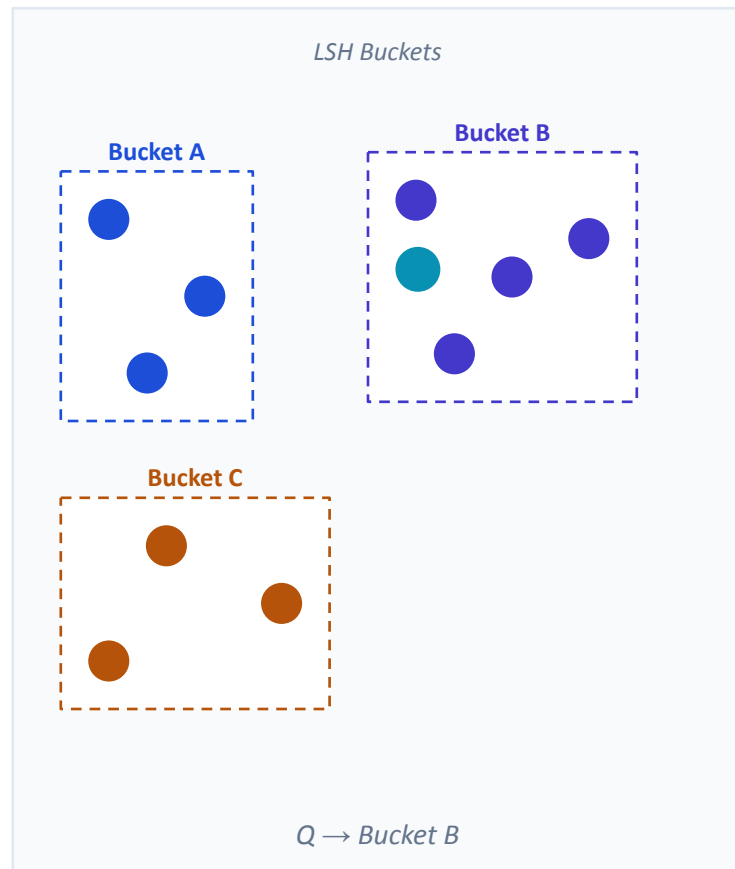
Sehr schnell bei Einfügen

Benötigt wenig RAM

Nachteile

Schlechtere Genauigkeit

Fehler bei dicht aneinander liegenden Daten



Filtered Search

Kombination von Metadaten-Filtern und Vektorähnlichkeit.

Beispiel: „Finde ähnliche Produkte wie dieses Bild — aber nur Produkte unter 50 € aus der Kategorie Schuhe.“

Zwei Strategien:

Pre-Filtering

1. Filter: Nur Schuhe unter 50 €
 2. Vektorsuche in dieser Teilmenge
- Schneller, da kleinerer Suchraum
→ Risiko: Teilmenge zu klein für gute Treffer

Post-Filtering

1. Vektorsuche über alle Daten
 2. Filter: Nur Ergebnisse unter 50 € behalten
- Vollständigere Ergebnisse
→ Teurer: Suche über den gesamten Datensatz

Möglich nur in einer Vektordatenbank mit MetadatenSpeicherung — nicht in einem reinen Index.

Full-Text Search: Hybrid-Suche

Keyword-Suche und semantische Vektorsuche kombiniert.

Beispiel: „Pizza Italien“ — exakt nach Keyword und semantisch nach ähnlichen Inhalten suchen.

Keyword-Suche (BM25)

Findet Dokumente, die den exakten Begriff „Pizza“ oder „Italien“ enthalten.

Präzise, aber kein Bedeutungsverständnis — „Neapolitanische Küche“ wird nicht gefunden.

Vektor-Suche (Semantisch)

Findet Dokumente mit ähnlicher Bedeutung — auch ohne die exakten Wörter.

„Italienische Küche“, „Margherita“, „Pasta“ werden als relevant erkannt.

Ergebnisse zusammenführen: RRF & Gewichtetes Scoring

Reciprocal Rank Fusion (RRF): Rankt Ergebnisse aus beiden Suchen zusammen, ohne Annahmen über Skalen.

Gewichtetes Scoring: Keyword-Treffer zählen X%, Vektor-Ähnlichkeit Y% — konfigurierbar je Anwendungsfall.

Ergebnis: Exakt + Semantisch = mehr relevante Treffer als beide Ansätze allein.



Was macht es zur „Datenbank“?

Index vs. vollständige Datenbankinfrastruktur

Die Ausgangsfrage

*„Ein Vektorindex kann ähnliche Vektoren finden,
aber was fehlt noch, damit es eine vollständige Datenbank ist?“*

- Ein Vektorindex ist ein mächtiges Werkzeug, aber kein vollständiges System
- Was unterscheidet das “Werkzeug” von der Infrastruktur?
- Was braucht eine Produktionsanwendung, das ein Index allein nicht liefert?

Was ein reiner Index leistet



Schnelle Ähnlichkeitssuche

ANN-Algorithmen finden ähnliche Vektoren in Millisekunden



Effiziente Algorithmen

HNSW, IVF und andere optimierte Suchstrukturen



Skaliert für große Mengen

Funktioniert für Millionen von Vektoren auf einer Maschine

Was einem reinen Index fehlt



Keine CRUD-Operationen

Kein Aktualisieren oder Löschen einzelner Einträge ohne kompletten Neuaufbau



Keine Metadatenstorage

Keine Filter wie „nur Kategorie X“ oder „nur Preis unter 50 €“



Keine Replikation / Fehlertoleranz

Fällt die Maschine aus, sind alle Daten verloren



Keine persistente Datenhaltung

Daten leben nur im RAM – kein Neustart möglich



Keine Zugriffskontrolle

Kein Multi-Tenancy, keine Rollenverwaltung

CRUD

Vektoren entstehen nicht einmalig und bleiben unverändert.



E-Commerce

Ein Nutzer aktualisiert seine Präferenzen. Ohne Update-Fähigkeit werden veraltete Empfehlungen geliefert.



Betrugserkennung

Neue Betrugsmerkmale müssen sofort eingepflegt werden. Veraltete Daten bedeuten nicht erkannten Betrug.



Dokumentensuche

Dokumente werden überarbeitet. Der alte Vektor muss ersetzt werden – ohne kompletten Index-Neuaufbau.



Löschen in der Praxis: Soft Deletes

Die meisten Implementierungen markieren Einträge als gelöscht und bereinigen sie periodisch — kein echtes In-Place-Löschen im Index.

Vollständige CRUD-Unterstützung ist kein Komfort – sie ist eine Grundanforderung für Produktionssysteme.

Das Metadaten-Problem: Warum Vektorsuche allein nicht reicht

Eine realistische Suchanfrage sieht nie so aus:

✗ „Finde die 10 ähnlichsten Vektoren im gesamten Datensatz.“

Sondern so:

✓ „Finde ähnliche Schuhe, aber nur unter 80 €, nur in Größe 42, nur auf Lager.“

✓ „Finde ähnliche Artikel, aber nur aus der Kategorie Outdoor, veröffentlicht nach 2023.“

✓ „Finde ähnliche Kandidatenprofile, aber nur mit Standort Berlin und min. 5 Jahren Erfahrung.“

Ohne Metadatenstorage ist keine dieser Anfragen möglich. Ein reiner Index kennt nur Vektoren.

Skalierung & Replikation

Horizontales Sharding

Daten werden auf mehrere Knoten verteilt – kein einzelner Flaschenhals

Replikation

Redundante Datenkopien für Ausfallsicherheit und höheren Lesedurchsatz

Unabhängige Skalierung

Suche, Indexierung und Einfügen skalieren separat nach Bedarf

Milliarden von Vektoren

Konsistente Performance auch bei extremem Datenvolumen

Ein Index läuft auf einer Maschine. Fällt sie aus, ist alles weg. Eine Datenbank verteilt und überlebt.

Zuverlässigkeit im Produktionsbetrieb

Was passiert, wenn etwas schief läuft?



Szenario: Knotenausfall

Ein Abfrageknoten fällt aus. In einem monolithischen System: Datenbankausfall. In einer Vektordatenbank: Die Replikationsschicht übernimmt automatisch. Kein Datenverlust, minimale Unterbrechung.



Szenario: Inkonsistente Daten

Daten werden gleichzeitig geschrieben und gelesen. Vektordatenbanken nutzen Konsistenzprotokolle wie Raft oder Paxos, um sicherzustellen, dass alle Knoten denselben Zustand sehen.

Reale Einsatzszenarien

Vektordatenbanken in der Produktion.

Unternehmen	Anwendungsfall	Benötigt DB-Features
Salesforce	Kundenunterstützung & Empfehlungen	CRUD, Filterung, Skalierung
PayPal	Betrugserkennung in Echtzeit	Echtzeit-Updates, Fehlertoleranz
eBay	Produktsuche & Ähnlichkeitsmatching	Metadaten-Filter, Millionen Einträge
Airbnb	Unterkunftsempfehlungen	CRUD, Multi-Tenancy
NVIDIA	KI-Infrastruktur	Skalierung, Replikation

Generelle Einsatzgebiete:



NLP

Textklassifikation,
Sentiment, Übersetzung



Computer Vision

Bilderkennung, autonomes
Fahren, Medizin



Genomik

Genetische Ähnlichkeiten,
Proteinstrukturen



Empfehlungen

Musik, Filme, Produkte,
Inhalte



Chatbots & KI

Kontextsuche, virtuelle
Assistenten

Fazit: Reiner Index vs. Vektordatenbank

Fähigkeit	Reiner Index	Vektordatenbank
Ähnlichkeitssuche (ANN)	✓	✓
CRUD-Operationen	✗	✓
Metadaten & Filterung	✗	✓
Persistente Datenhaltung	✗	✓
Fehlertoleranz & Replikation	✗	✓
Horizontal skalierbar	✗	✓
Produktionstauglich	✗	✓

Für welche eigene App-Idee oder welchen Anwendungsfall würdet ihr eine Vektordatenbank einsetzen?

Denkanstoß: Was wäre der konkrete Mehrwert gegenüber einer klassischen Datenbanksuche?



Ein Vektorindex findet Ähnlichkeit.

Eine Vektordatenbank verwaltet Wissen.

Vektordatenbanken sind keine Nischenlösung.

Sie sind vollwertige Datenbankinfrastruktur – gebaut für die KI-Ära.